[PASSED] Case study

F&B retail · Build + test

Four outlets, one spreadsheet.

A multi-outlet point-of-sale system for a dimsum business, built on Google Apps Script and Google Sheets, and tested end to end.

1 · Analysis | 2 · Requirements | 3 · Database design | 4 · Build | 5 · Testing

Mulla Shadra · QA Engineer · Client and product names anonymized, shared with the owner’s permission

01 · The business

A dimsum business run from one central kitchen

4 + 1
outlets selling, one kitchen producing everything

~320
transactions a day across the outlets

7
user accounts in four roles

0
printers: cashiers work from a phone

> The owner is not technical. The system had to be easy to use, cheap to run, and keep its data somewhere he could open and read himself, without calling a developer.

02 · Starting point

The old sheet worked, until you looked closely

Passwords in plain text

All seven accounts were written directly in the script.

Open to anyone with the link

The spreadsheet could be opened, and even edited, by anyone holding its URL.

A cashier page that couldn’t sell

It called functions that didn’t exist, used dummy data and had no guard against double submits.

Half-written transactions

A bug could stop a sale midway, leaving partial data behind.

03 · Requirements

Six things the business needed

01

Fast checkout on a phone

Cashiers sell from a phone, with a digital receipt.

02

Controlled ingredient stock

Raw materials tracked at the central kitchen.

03

Profit per outlet

Not just revenue: cost of goods and fixed costs too.

04

Prices per sales channel

Walk-in, GoFood, GrabFood and ShopeeFood each priced separately.

05

Approved voids and refunds

Requested by the cashier, approved by the owner.

06

Access by role and branch

Each user sees only what their job and outlet allow.

04 · Platform choice

Why Google Apps Script and Google Sheets

Zero running cost	No server, domain or paid hosting.
Data the owner knows	He can open, filter and export Sheets himself.
No migration	The business already ran on Google accounts.
Just a link	Staff open it in a phone browser; nothing to install.
Enough for the scale	~320 sales a day is far below Sheets' limits.

Limits, managed from day one

- Slower than a real database: checkout takes about 5–6 s, about 10–12 s when two cashiers pay at once.
- No database transactions, so concurrent writes are serialized with LockService.
- Execution quotas and the 10-million-cell cap: archive every six months.
- One data-access layer, so moving to MySQL or Firebase means rewriting one file.

05 · Architecture

One entry point, one data layer

BROWSER

Phone or laptop

Plain HTML, CSS and JS. Every call goes through api(route, token, data).



APPS SCRIPT SERVER

`Web.gs`	single entry, route whitelist
`Auth.gs`	login, session, role and branch guard
`*Service.gs`	business rules per module
`Database.gs`	the only code that touches Sheets



DATABASE

Google Sheets

27 sheets: master data, sales, stock, purchasing, audit.

Rules live on the server. Prices, totals, cost and access are recalculated there; the browser is never trusted.

Columns read by header name, so the owner can reorder a sheet without breaking anything.

Safe write order: stock, lines, payment, then the header last. A failure midway rolls stock back.

06 · Users

Four roles, each with only what it needs

Cashier

4 accounts · 1 per outlet

Pick the channel, build the cart, pay by cash, QRIS or online, share a digital receipt, request a void.

Owner

Full control

Dashboard, menu and recipes, channel prices, promos, users, void and purchase approvals, audit log.

Finance

Reports

Sales and profit per outlet, plus expenses when the owner allows it.

Inventory

Kitchen and purchasing

Ingredient stock, stock in, stock requests, purchase records, stock movement.

Mobile-first: every screen tested at phone width (390 px) and laptop width (1280 px).

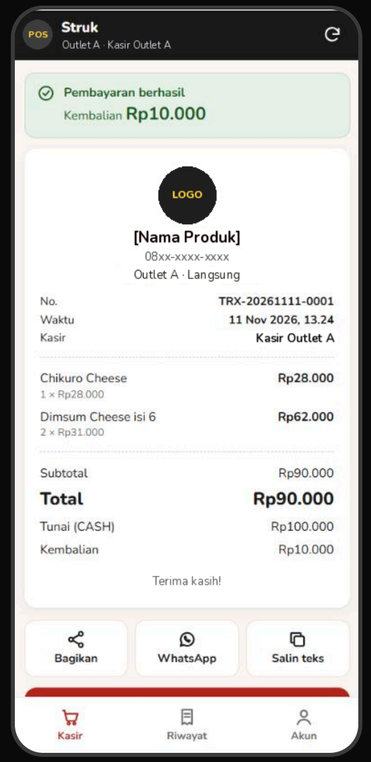
07 · The app

From cart to receipt on one phone

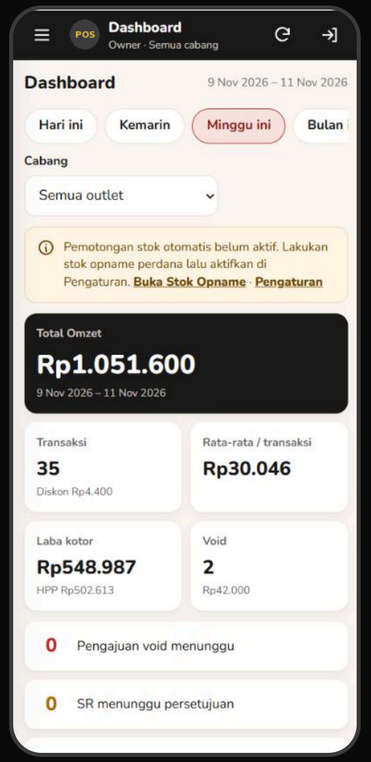
Real screens from the test database. Brand, logo, contacts and outlet names are masked.



1 · Cashier builds the cart



2 · Digital receipt, no printer



3 · Owner sees it on the dashboard

08 · Business logic

Every sale deducts raw ingredients

A two-level bill of materials links the menu to the kitchen.



Components are semi-finished items plus packaging. Cost of goods updates by itself when an ingredient price or a recipe changes.

PROFIT PER OUTLET

revenue – cost of goods – share of fixed costs – expenses

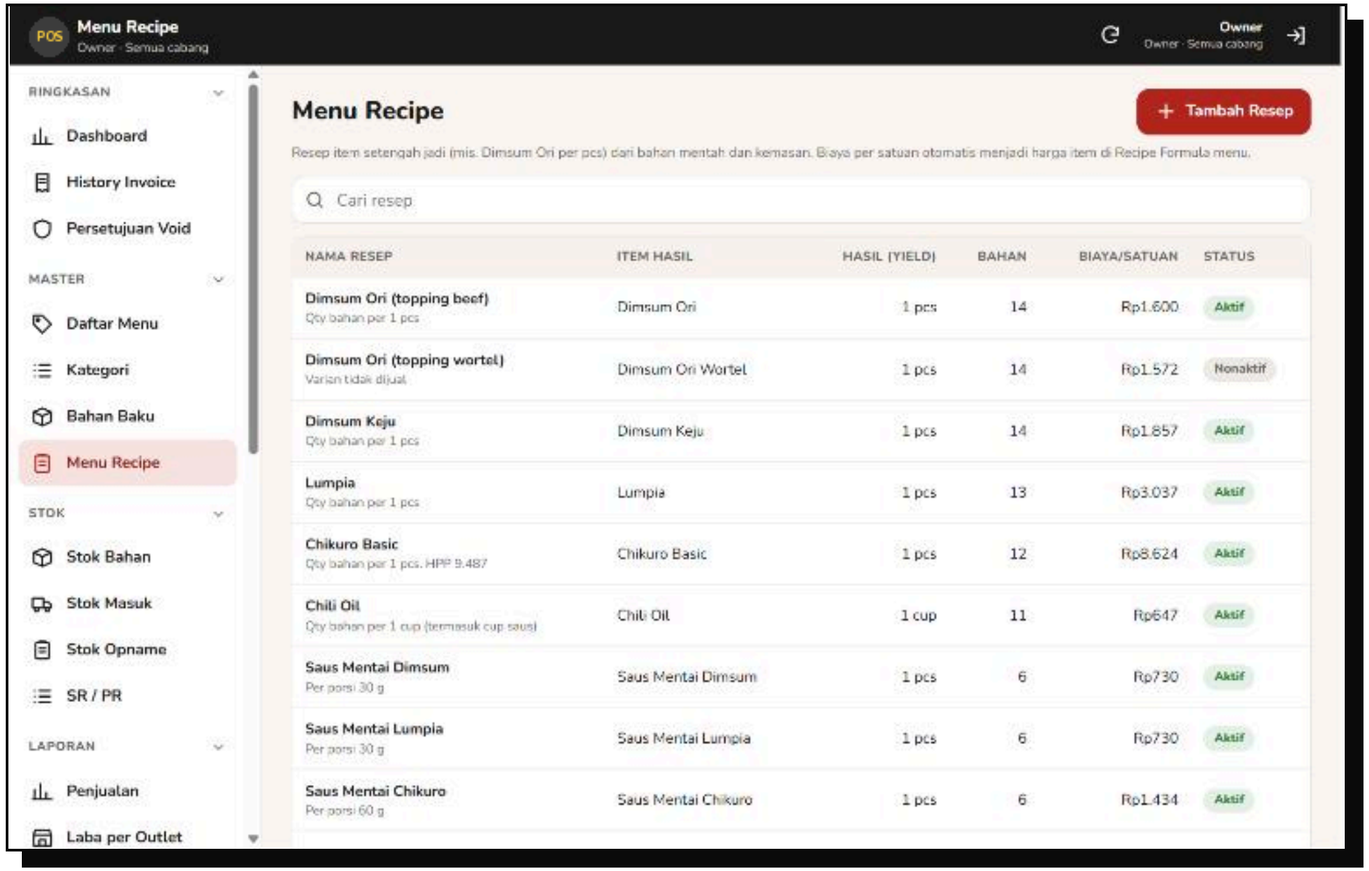
VERIFIED IN TESTING

1 Dimsum Mentai (6 pcs) deducts exactly 28 ingredients

Ingredient	Deducted
Chicken	120 g
Dimsum wrappers	6
Mayonnaise	132 g
Oil	17.24 ml
Box, cup, chopsticks, bag	1 each

A sale that fails on low stock leaves no data behind.

The recipe model, as the owner sees it



Recipes per piece

Each semi-finished item lists its raw ingredients and packaging.

Cost updates itself

Change an ingredient price and the cost per unit follows.

Feeds the profit report

Unit costs roll up into menu cost and profit per outlet.

09 · Security and data integrity

Serious safeguards on a simple platform

Hashed passwords

SHA-256, 100 rounds, a unique salt per user. Forced change on first login.

Lockout

15 minutes after 5 wrong tries; errors never reveal whether a username exists.

Server-side guards

Role and branch checked in every function; a cashier is pinned to their own outlet.

No double sales

A unique request ID per cart: a resend after a dropped signal returns the old receipt.

XSS prevention

All data rendered through escaping or textContent.

Audit log

Logins, sales, price changes, voids and stock counts are all recorded.

10 · Process

Signed off phase by phase

0 · Done

Analysis

Legacy review, risks, permission matrix

1 · Approved

Requirements

Software requirements specification

2 · Approved

Database design

27 sheets, statuses, purchasing flow

3 · Done

Sheet prep

Backup, new columns, costs recalculated

4 · Done

Backend

22 .gs files: auth, services, reports, audit

5 · Approved

Frontend

Three sample screens approved first

6 · Done

Integration and testing

0 functional bugs at the end

7 · Approved

Client UAT

System and flows approved, minor fixes only

> Every decision was logged, and any change to the client’s data or sheet structure was explained and approved first.

11 · Testing

Proven before the client’s UAT, not during it

421

automated checks, all passing, plus a concurrency test with two cashiers paying at once.

Suite	Result
Local Node.js simulation, Apps Script mocked	129 / 129
Playwright end to end, all roles, 390 and 1280 px	188 / 188
Read-only checks on the live server	21 / 21
Write checks on the live server	83 / 83
10 parallel sales from two cashiers	Pass

Write checks cover cash, QRIS and online payments, channel prices, promos, voids, ingredient deduction, stock counts, purchasing, access rights, lockout, session expiry and XSS. Parallel sales kept transaction numbers unique and in order, and stock dropped by exactly the right amount.

12 · Lessons

What the project taught me

Fix before you rebuild	Keep what the legacy code already did right; replace only what was unsafe.
Test without breaking data	Flag test records, log every temporary switch, restore everything automatically.
Prove concurrency, don’t assume it	Two server executions run side by side showed the lock and numbering hold.
Measure, then report it	Checkout at 5–6 s was flagged to the client openly and scheduled for tuning.

✓ Where it stands

Built, tested, client-approved.

My first development project, built with Claude as an AI pair programmer. I review and verify every output myself, and the testing approach comes straight from my QA work.

mshadra6@gmail.com

linkedin.com/in/mulla-shadra

mullash.github.io/Mulla-Portfolio